

tf.numpy_function Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/numpy_function

Wraps a python function and uses it as a TensorFlow op.

Function Signatures

```
tf.numpy_function( func=None, inp=None, Tout=None,  
stateful=True, name=None )
```

Code Examples

```
tf.numpy_function(  
    func=None, inp=None, Tout=None, stateful=True, name=None  
)
```

```
tf.numpy_function(  
    func=None, inp=None, Tout=None, stateful=True, name=None  
)
```

```
@tf.numpy_function(Tout=tf.float32)  
def my_numpy_func(x):  
    # x will be a numpy array with the contents of the input to the  
    # tf.function  
    print(f'executing eagerly, {x=}')  
    return np.sinh(x)
```

```
@tf.numpy_function(Tout=tf.float32)
```

```
def my_numpy_func(x):
```

```
# x will be a numpy array with the contents of the input to the
```

```
# tf.function
```

```
print(f'executing eagerly, {x=}')
```

Documentation

Wraps a python function and uses it as a TensorFlow op.

View aliases

Compat aliases for migration

See

Migration guide for
more details.

`tf.compat.v1.numpy_function`

Used in the notebooks

- Playing CartPole with the Actor-Critic method

Given a python function `func` wrap this function as an operation in a `tf.function`. `func` must take numpy arrays as its arguments and return numpy arrays as its outputs.

There are two ways to use `tf.numpy_function`.

As a decorator

When using `tf.numpy_function` as a decorator:

- you must set `Tout`
- you may set `name`
- you must not set `func` or `inp`

The function runs eagerly:

The behavior doesn't change inside a `tf.function`:

Inplace

This form can be useful if you don't control the function's source, but it is harder to read.

Here is the same function with no decorator:

To run `tf.numpy_function` in-place, pass the function, its inputs, and the output type in a single call to `tf.numpy_function`:

More info

Comparison to `tf.py_function`:

`tf.py_function` and `tf.numpy_function` are very similar, except that `tf.numpy_function` takes numpy arrays, and not `tf.Tensors`. If you want the function to contain `tf.Tensors`, and have any TensorFlow operations executed in the function be differentiable, please use `tf.py_function`.

- Calling `tf.numpy_function` will acquire the Python Global Interpreter Lock

(GIL) that allows only one thread to run at any point in time. This will preclude efficient parallelization and distribution of the execution of the program. Therefore, you are discouraged to use `tf.numpy_function` outside of prototyping and experimentation.

- The body of the function (i.e. `func`) will not be serialized in a

`tf.SavedModel`. Therefore, you should not use this function if you need to serialize your model and restore it in a different environment.

- The operation must run in the same address space as the Python program

that calls `tf.numpy_function()`. If you are using distributed TensorFlow, you must run a `tf.distribute.Server` in the same process as the program that calls `tf.numpy_function` you must pin the created operation to a device in that server (e.g. using with `tf.device():`).

- Currently `tf.numpy_function` is not compatible with XLA. Calling

`tf.numpy_function` inside `tf.function(jit_compile=True)` will raise an error.

- Since the function takes numpy arrays, you cannot take gradients

through a `numpy_function`. If you require something that is differentiable, please consider using `tf.py_function`.

Calling `tf.numpy_function` will acquire the Python Global Interpreter Lock (GIL) that allows only one thread to run at any point in time. This will preclude efficient parallelization and distribution of the execution of the program. Therefore, you are discouraged to use `tf.numpy_function` outside of prototyping and experimentation.

The body of the function (i.e. `func`) will not be serialized in a `tf.SavedModel`. Therefore, you should not use this function if you need to serialize your model and restore it in a different environment.

The operation must run in the same address space as the Python program that calls `tf.numpy_function()`. If you are using distributed TensorFlow, you must run a `tf.distribute.Server` in the same process as the program that calls `tf.numpy_function` you must pin the created

operation to a device in that server (e.g. using with `tf.device():`).

Currently `tf.numpy_function` is not compatible with XLA. Calling `tf.numpy_function` inside `tf.function(jit_compile=True)` will raise an error.

Since the function takes numpy arrays, you cannot take gradients through a `numpy_function`. If you require something that is differentiable, please consider using `tf.py_function`.

Returns

- If `func` is `None` this returns a decorator that will ensure the decorated function will always run with eager execution even if called from a `tf.function/tf.Graph`.

- If used `func` is not `None` this executes `func` with eager execution

and returns the result: A single or list of `tf.Tensor` which `func` computes.